

HPC Filesystem & Packed Data Format

Bryan Cardenas

Lars Veefkind

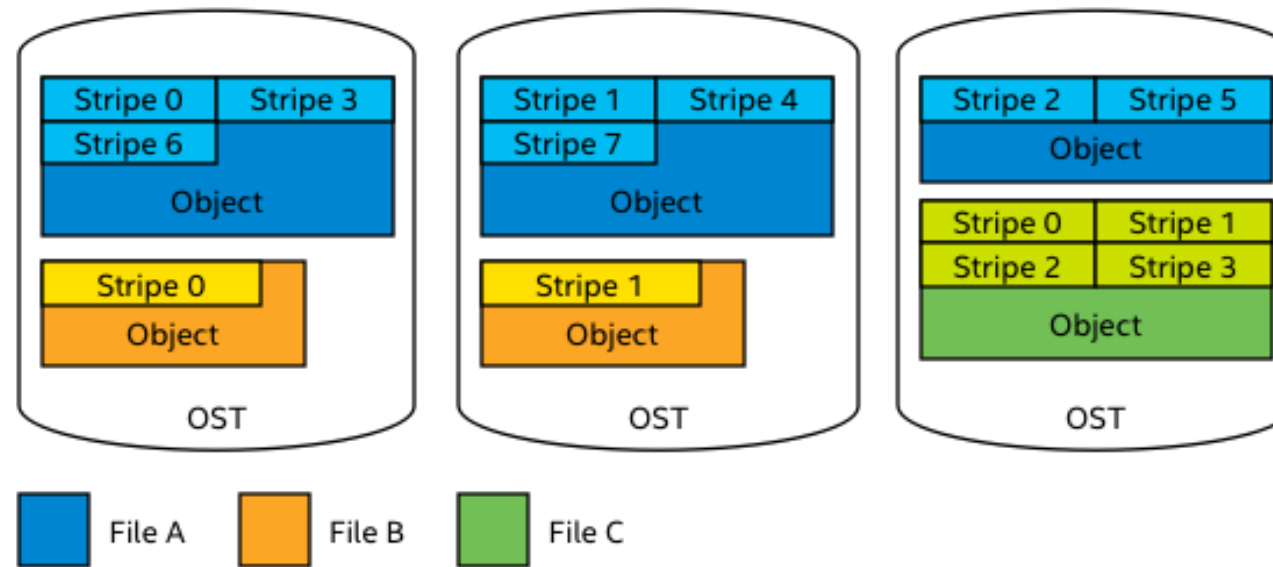
High Performance Machine Learning

SURF

HPC File System

HPC systems typically use parallel shared file systems.

- **Parallel file system:** one file can be distributed over many physical disks, to increase I/O bandwidth.



HPC File System

Two main types of parallel filesystems

- **Lustre**
 - Metadata (filename, size, location, etc) stored on separate server
 - Object Storage Target (OST) stores actual file
 - Striping over multiple OSTs can be managed by user
- **GPFS (Snellius)**
 - Metadata and actual file stored on the same server
 - Striping is managed automatically, by file system. User has no control.

HPC File System

HPC file systems are optimized for ***bandwidth*** not I/O operations per second (IOPS)

- E.g. Snellius GPFS: up to 10 GB/s write, 20 GB/s read
- Important when e.g. a 100 node job needs to start from the same 20 GB input file...

HPC file systems are typically **not** good at AI / data science workloads

- AI workloads are generally: reading lots of small files / parts of a file

HPC File System

- An inode stores all information about a linux file *except* its name and data
 - File size, last change date, file permissions, etc
- On most HPC systems, you'll have an inode quota, in addition to a size quota
 - Like total size, each filesystem has a (fixed) maximum number of inodes
 - It is *also* a way to discourage doing too many IOPS

HPC File System

- An inode stores all information about a linux file *except* its name and data
 - File size, last change date, file permissions, etc
- On most HPC systems, you'll have an inode quota, in addition to a size quota
 - Like total size, each filesystem has a (fixed) maximum number of inodes
 - It is *also* a way to discourage doing too many IOPS

Seems we'll run out of inodes before we run out of space on e.g. our home1 filesystem:

```
$ df -ih /gpfs/home1/
```

Filesystem	Inodes	IUsed	IFree	IUse%	Mounted on
home1	287M	161M	126M	57%	/gpfs/home1

```
$ df -h /gpfs/home1/
```

Filesystem	Size	Used	Avail	Use%	Mounted on
home1	121T	62T	59T	52%	/gpfs/home1

HPC File System

Suppose your dataset is 10k png files of 2k each...

- Home1: 121TB, 287M inodes, so 421k per inode
 - If everyone does this, the filesystem will be full of inodes, with 99.5% / 120 TB (!) storage still available...

```
-rw-r--r-- 1 1.8K 1000.png
-rw-r--r-- 1 2.3K 1001.png
-rw-r--r-- 1 2.1K 1002.png
-rw-r--r-- 1 2.6K 1003.png
-rw-r--r-- 1 2.7K 1004.png
-rw-r--r-- 1 2.0K 1005.png
-rw-r--r-- 1 2.4K 1006.png
-rw-r--r-- 1 2.3K 1007.png
-rw-r--r-- 1 2.6K 1008.png
-rw-r--r-- 1 2.1K 1009.png
```

•
•
•

```
-rw-r--r-- 1 2.3K 9990.png
-rw-r--r-- 1 1.6K 9991.png
-rw-r--r-- 1 2.5K 9992.png
-rw-r--r-- 1 2.5K 9993.png
-rw-r--r-- 1 2.7K 9994.png
-rw-r--r-- 1 2.1K 9995.png
-rw-r--r-- 1 2.1K 9996.png
-rw-r--r-- 1 2.7K 9997.png
-rw-r--r-- 1 2.3K 9998.png
-rw-r--r-- 1 2.2K 9999.png
-rw-r--r-- 1 2.1K 999.png
```

HPC File System

Suppose your dataset is 10k png files of 2k each...

- Home1: 121TB, 287M inodes, so 421k per inode
 - If everyone does this, the filesystem will be full of inodes, with 99.5% / 120 TB (!) storage still available...
- If you run 100 epochs, you'll do 1M I/O operations
 - Your sysadmin won't like you
 - Your fellow users won't like you: everyone suffers slow I/O because of your job!
 - You shouldn't like you: your I/O might be holding back your training performance...

```
-rw-r--r-- 1 1.8K 1000.png
-rw-r--r-- 1 2.3K 1001.png
-rw-r--r-- 1 2.1K 1002.png
-rw-r--r-- 1 2.6K 1003.png
-rw-r--r-- 1 2.7K 1004.png
-rw-r--r-- 1 2.0K 1005.png
-rw-r--r-- 1 2.4K 1006.png
-rw-r--r-- 1 2.3K 1007.png
-rw-r--r-- 1 2.6K 1008.png
-rw-r--r-- 1 2.1K 1009.png
```

•
•
•

```
-rw-r--r-- 1 2.3K 9990.png
-rw-r--r-- 1 1.6K 9991.png
-rw-r--r-- 1 2.5K 9992.png
-rw-r--r-- 1 2.5K 9993.png
-rw-r--r-- 1 2.7K 9994.png
-rw-r--r-- 1 2.1K 9995.png
-rw-r--r-- 1 2.1K 9996.png
-rw-r--r-- 1 2.7K 9997.png
-rw-r--r-- 1 2.3K 9998.png
-rw-r--r-- 1 2.2K 9999.png
-rw-r--r-- 1 2.1K 999.png
```


Packed Data Format

What is a 'packed data format'?

- Group multiple individual files together in a single 'packed'/archive file

```
-rw-r--r-- 1 1.8K 1000.png  
-rw-r--r-- 1 2.3K 1001.png  
-rw-r--r-- 1 2.1K 1002.png  
-rw-r--r-- 1 2.6K 1003.png  
-rw-r--r-- 1 2.7K 1004.png  
-rw-r--r-- 1 2.0K 1005.png  
-rw-r--r-- 1 2.4K 1006.png  
-rw-r--r-- 1 2.3K 1007.png  
-rw-r--r-- 1 2.6K 1008.png  
-rw-r--r-- 1 2.1K 1009.png
```

⋮

```
-rw-r--r-- 1 2.3K 9990.png  
-rw-r--r-- 1 1.6K 9991.png  
-rw-r--r-- 1 2.5K 9992.png  
-rw-r--r-- 1 2.5K 9993.png  
-rw-r--r-- 1 2.7K 9994.png  
-rw-r--r-- 1 2.1K 9995.png  
-rw-r--r-- 1 2.1K 9996.png  
-rw-r--r-- 1 2.7K 9997.png  
-rw-r--r-- 1 2.3K 9998.png  
-rw-r--r-- 1 2.2K 9999.png  
-rw-r--r-- 1 2.1K 999.png
```



```
-rw-r--r-- 1 22M archive.tar
```

Packed Data Format

What is a 'packed data format'?

- Group multiple individual files together in a single 'packed'/archive file
- Examples of packed data formats → ZIP, TAR, LMDB, HDF5, PARQUET, zarr, etc.
- Does not necessarily involve compression!
 - Compression is possible, but often runs on single process, so can be slow to decompress



Packed Data Format

What is a 'packed data format'?

- Group multiple individual files together in a single 'packed'/archive file
- Examples of packed data formats → ZIP, TAR, LMDB, HDF5, PARQUET, etc.
- Does not necessarily involve compression!
 - Compression is possible, but often runs on single process, so slow to decompress

Why packed data format

HPC systems use a **shared** filesystem. Packed data formats...

- Reduces inode consumption
- Reduces number of I/O operations/s
 - Reduces slowdown for yourself *and* other users

<https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/56295498/Best+Practice+for+Data+Formats+in+Deep+Learning>

| Local Scratch Disks

Some HPC systems have **local disks** *in* the nodes

- On Snellius, these can be requested by adding `#SBATCH -constraint=scratch-node` to your batch script
- Note: about half of our A100 and $\frac{1}{4}$ of our H100 nodes (currently reserved) have local scratch disk. You might be in the queue longer if you request `scratch-node`, so only do it if you think you benefit from it
- H100s have 7TB, A100 5.6TB of local scratch

| Local Scratch Disks

Some HPC systems have **local disks** *in* the nodes

- On Snellius, these can be requested by adding `#SBATCH -constraint=scratch-node` to your batch script
- Note: about half of our A100 and $\frac{1}{4}$ of our H100 nodes (currently reserved) have local scratch disk. You might be in the queue longer if you request `scratch-node`, so only do it if you think you benefit from it
- H100s have 7TB, A100 5.6TB of local scratch
- Local scratch disks can usually handle much more IOPs
- Local scratch disks are **shared** between all users in a node
- Users on the same node *may* still experience some effects when you ‘hammer’ local scratch with I/O, *but it’s way less bad than on a shared filesystem!*

Local Scratch Disks

Some HPC systems have local disks *in* the nodes

- If you really need to use *individual* files instead of a packed file format, use local scratch disks in the following way:

1. **Store** zip/tar on shared parallel Filesystem.
2. At start of job: **extract** to local scratch dir (for every node, if you use multiple)
3. **Do** any further I/O from that local scratch dir

Local RAM Disks

All Snellius nodes have system RAM (duhh)

- You can access this as a ramdisk in `/dev/shm` (shared memory)
 - RAM is extremely fast (bandwidth & IO)
 - +- 25GB/s (A100) to 38GB/s
 - Orders of magnitude better IOPS
 - If your data fits on RAM, there is no reason not to load your data there before training
 - H100: 768GB/node, 192GB/GPU
 - A100: 512GB/node, 128GB/GPU
1. **Store** zip/tar on shared parallel Filesystem.
 2. At start of job: **extract** to `/dev/shm` (for every node, if you use multiple)
 3. **Do** any further I/O from `/dev/shm`

| Want to learn more?

More info

<https://servicedesk.surf.nl/wiki/display/WIKI/Best+Practice+for+Data+Formats+in+Deep+Learning>

<https://github.com/sara-nl/Packed-Data-Formats>

DEMO

| **Using and benchmarking a packed file format dataloader**